ONTARIO TECH UNIVERSITY
FACULTY OF SCIENCE, COMPUTER SCIENCE

CSCI 4220U Computer Vision – Course Project April 10, 2026

Seeing Sound: Computer Vision-Guided Spectrogram Watermarking for Audio–Visual Fusion

by

Tobin Kellock (100472987)

Abstract: Spectrograms convert audio into an image-like representation (time $\times$ frequency), making them a natural meeting point between signal processing and computer vision. This project extends an earlier "image-to-sound" prototype by introducing spectrogram watermarking: given a carrier audio clip and an input image, the program embeds a visually meaningful watermark into the carrier's magnitude spectrogram while preserving phase for stable reconstruction. The pipeline uses classic computer vision operations such as grayscale normalization, gradient-based edge extraction, thresholding, morphological dilation, smoothing, resizing, and geometric flips to create a robust mask that remains visible after mapping onto the STFT grid. The result is a single WAV file that retains the character of the original audio to a tunable degree while producing a spectrogram that reveals the embedded image.

Keywords: spectrograms, STFT, audio watermarking, audio–visual fusion

1 Introduction

A spectrogram is a visual summary of how energy in an audio signal is distributed across frequency over time. Because spectrograms are images, many computer vision ideas (filtering, edge emphasis, contrast control, geometric transforms) can be applied to improve how information can be displayed or embedded.

The first version of this project explored the sonification of images: mapping pixel intensities into a frequency-domain representation, synthesizing a waveform, and then visualizing the result as a spectrogram. The updated version aims for a more realistic and more interesting task: audio–visual fusion. Given an image and an audio file, the program edits the audio's spectrogram so the spectrogram "shows" the image like a watermark, while still sounding like the carrier audio as much as possible.

This report includes Related Work, Methodology, Results, Discussion and Conclusions. A link to the projects code repository is also included.

2 Related Work

Spectrograms and STFT; The Short-Time Fourier Transform (STFT) is a standard tool for analyzing non-stationary signals. By applying a windowed Fast Fourier Transform (FFT) over overlapping frames, it produces a time–frequency representation that can be plotted as an image.

Phase-aware reconstruction; Editing only the magnitude of the spectrogram and reusing, or iteratively estimating, the phase is a common practical approach. Phase is essential for high-quality inversion; the Griffin–Lim algorithm is a classic method to estimate phase when only the magnitude is available.

Audio watermarking; Traditional audio watermarking embeds information in ways designed to be robust to transformations and minimally perceptible. In this project, the goal is intentionally visible watermarking in the spectrogram domain: the mark should be visually apparent, with a tunable trade-off between visibility and audio fidelity.

3 Methodology

The final project includes three key artifacts: the baseline image-to-audio synthesizer (`spectrogram.py`), the watermarking system (`spectrogram_watermark.py`), and the visualization script (`visualize.py`).

3.1 Version 1 Baseline: Image to Sound, Sound to Spectrogram

The baseline converted an input image into a waveform by treating each image row as frequency content and applying an inverse real FFT. The image is converted to grayscale, transposed to align axes with (time and frequency), and vertically flipped so the final spectrogram matches the original image orientation. A non-linear intensity transform increases contrast before synthesis.

3.2 Version 2: Spectrogram Watermarking

The watermarking pipeline builds upon the first version by taking a carrier WAV file and an image and produces a new WAV file whose spectrogram reveals the image. The program pipeline is as follows:

1. **Load carrier audio:** Read 16-bit PCM WAV, convert to mono if needed, and normalize samples to $[-1, 1]$.

2. **Create a watermark mask:** Convert the image to grayscale and normalize to $[0, 1]$. Then apply one of two mask options:

   - **Luma mask:** use the grayscale image directly

   - **Edge mask:** compute image gradients (via `np.gradient`), take gradient magnitude as an edge map, threshold to keep strong edges, and apply morphological dilation, max filter, to thicken edges.

   The mask may also be blurred, using low-pass filtering, to reduce harsh transitions, then resized using bilinear interpolation to match the STFT grid.

3. **Compute STFT:** Frame the audio with a Hann window of length N (`nfft`) and hop H (`hop`), compute $X(t, k) = \mathrm{FFT}(x_t)$. This yields magnitude $|X|$ and phase $\angle X$.

4. **Embed watermark into magnitude:** The program preserves phase and edits magnitude in the log domain for perceptual stability:

$$\log|\tilde{X}(t,k)| = \log|X(t,k)| + \Delta(t,k).$$

   The update Δ depends on a chosen embedding mode:

   - **Overlay mode:** small additive perturbation for better audio fidelity.
   - **Blend mode:** blend the carrier toward an image-shaped target magnitude for stronger visibility.

   Band-limiting weights (`fmin, fmax`) and a smooth taper restrict where modifications occur. Optional flips (`--flip_freq`, `--flip_time`) correct orientation to allow the spectrogram to maintain image orientation for better matching.

5. **Reconstruct audio using inverse STFT:** Combine edited magnitude with original phase and apply inverse FFT per frame, then overlap-add frames. A window-energy normalization and peak normalization prevent clipping.

6. **Visualize:** `visualize.py` generates the final spectrogram image, with dB scaling and a fixed visible range for consistency.

3.3 Computer vision techniques used

Although the final output is a spectrogram, the watermark is produced by applying computer vision operations to the input image:
Grayscale conversion and normalization; Gradient-based edge extraction; Thresholding using quantiles; Morphological dilation via max filtering; Smoothing (separable blur) to reduce speckle; Bilinear resizing to match the time–frequency grid; Geometric flips to match spectrogram orientation.

4 Results

This section shows the qualitative results on a sample image (`Cat.jpg`) and its spectrogram outputs. Two outputs are shown: the baseline image-to-audio spectrogram, and the watermarked carrier-audio spectrogram.

Figure 1: Input image, `Cat.jpg`.

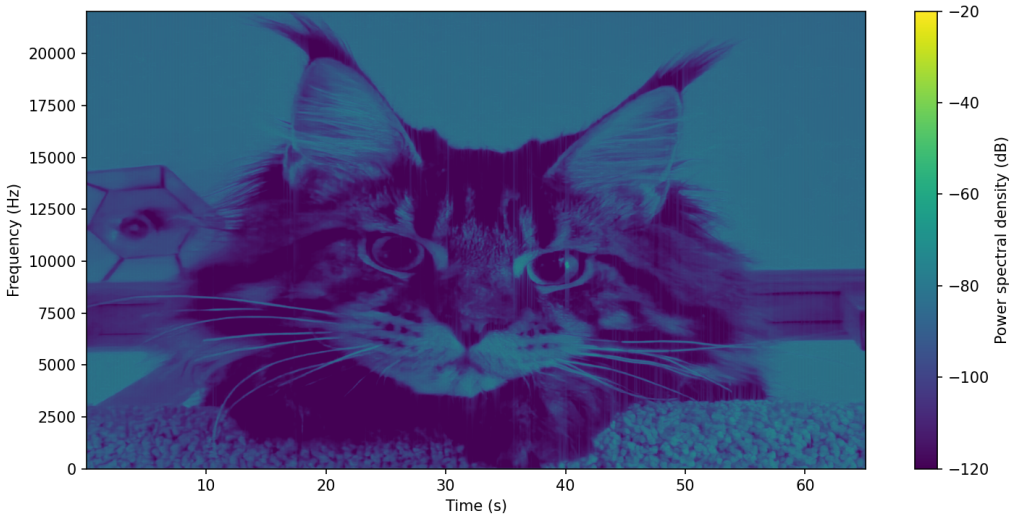

Figure 2: Baseline result: spectrogram generated from image-to-audio synthesis.

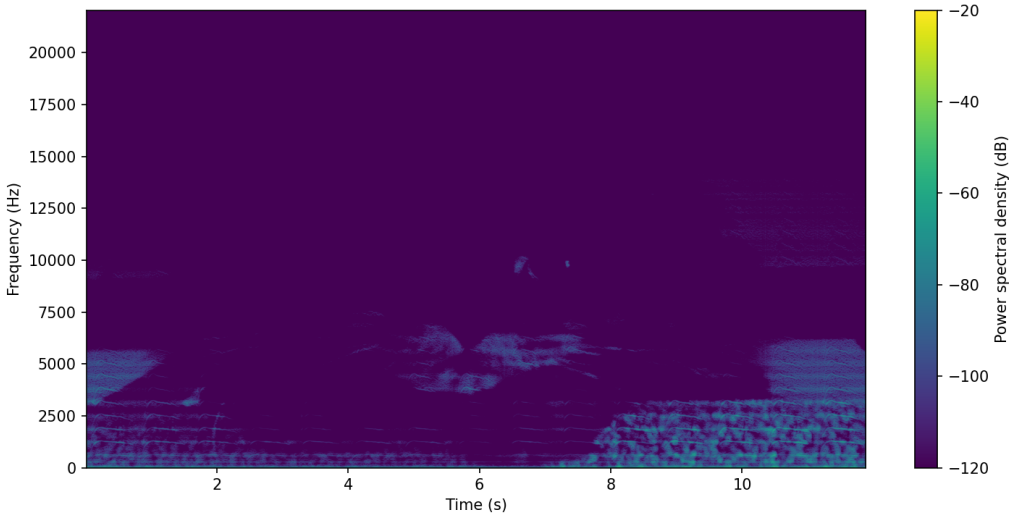

Figure 3: Watermarked result: carrier audio edited so its spectrogram reveals the embedded image pattern, stronger settings increase visibility at the cost of more audio change.

4.1 Parameter settings

The watermark system uses parameters that control the audio–image trade-off to control the image and audio clarity. Table 1 lists some of the options.

Parameter	Effect
`nfft`, `hop`	Time–frequency resolution; smaller hop increases image "width" adding more frames.
`mask` (luma/edges)	Luma preserves shading, edges improves readability under limited resolution.
`edge_dilate`, `blur_passes`	Morphology and smoothing: strengthen thin structures vs. reduce artifacts.
`mode` (overlay/blend)	Overlay keeps audio clearer; blend makes the image more visible.
`mix`, `depth_db`	In blend mode: increases how strongly the spectrogram follows the image.
`fmin`, `fmax`	Band limiting: restrict watermark to a frequency range, helps to controls audibility/visibility.
`--flip_freq`, `--flip_time`	Geometric corrections when spectrogram orientation differs from the input image.

Table 1: Key parameters used to tune the watermark visibility vs. audio fidelity trade-off.

5 Discussion and Conclusions

Trade-offs: The main trade-off is between image visibility and audio fidelity. Increasing blend strength (`mix`) and contrast (`depth_db`) makes the watermark clearer but changes the sound more. Restricting the watermark to a band (`fmin/fmax`) can preserve key audible regions while still showing the image.

Why edges often work better: Real-world photos contain texture that becomes difficult to represent at limited STFT resolution. Gradient-based edges and dilation produce sparse, high-contrast features that survive resizing and remain visible in the spectrogram.

Limitations: This approach intentionally prioritizes spectrogram appearance. The current implementation does not optimize a perceptual audio objective, and the spectrogram visibility depends on chosen display scaling. Highly tonal carrier audio can still introduce strong horizontal patterns that compete with the watermark.

Future work: Two natural extensions are perceptual masking to embed more strongly where changes are less audible, and learning-based post-processing (e.g. using CNN, LSTM, and Transformer) to enhance watermark clarity under compression or noise.

A Appendix: Code and Artifacts

Code: The project code can be found in the following repository link. The repository includes: `spectrogram.py` (the original spectrogram generation program that this project was built upon), `spectrogram_watermark.py` (the new watermarking program that was developed for this project), and `visualize.py` (spectrogram rendering program, it was updated to function with both versions of spectrogram generation).

Repository link:

https://github.com/Tobin360/Seeing-Sound-Project